

A crossed random-effects latent class model for annotation ensembles of large language models

1*

1* .

Abstract

Large language models (LLMs) are increasingly used as annotators, but every annotation is produced by a configuration of model, prompt and stochastic decoding run, and configurations that share training corpora and alignment procedures do not err independently. [ABSTRACT: 150–250 words, no citations; to be finalised with simulation results.]

Keywords: latent class model, crossed random effects, Dawid–Skene, large language models, text annotation, conditional dependence

MSC Classification: 62H30 , 62F15 , 62J12 , 62P25 , 68T50

1 Introduction

2 Background and related work

3 Model

4 Simulation study

5 Application

6 Discussion

Supplementary information. Stan code for all model variants, additional tables and the full per-cell simulation results are provided as Online Resource 1. The code repository is available through an anonymised link for review.

Appendix A Stan code

```
// crossed_lcre.stan
// Binary latent class model with crossed random effects for
// LLM annotation ensembles (model x prompt x run).
// The discrete class c_i is marginalised out.

data {
  int<lower=1> N; // items
  int<lower=1> M; // LLM models
  int<lower=1> P; // prompts
  int<lower=1> R; // runs per (model, prompt)
  array[N, M, P] int<lower=0, upper=R> S; // # runs labelled 1
}

parameters {
  real<lower=0, upper=1> pi1; // prevalence of class 1
  ordered[2] mu; // class intercepts (label
    identification)
  matrix[M - 1, 2] a_raw; // model effects, model 1
    = reference
  matrix[P, 2] b_z; // prompt effects (non-
    centred)
  vector<lower=0>[2] tau_b;
  real<lower=0> s_th; // item ambiguity SD
  real<lower=0> s_ph; // item x model SD
  real<lower=0> s_ps; // item x prompt SD
  vector[N] th_z;
  matrix[N, M] ph_z;
  matrix[N, P] ps_z;
}

transformed parameters {
  matrix[M, 2] a;
  matrix[P, 2] b;
  a[1] = rep_row_vector(0, 2);
  if (M > 1) a[2:M] = a_raw;
  for (k in 1:2) b[, k] = tau_b[k] * b_z[, k];
}

model {
  // priors
  pi1 ~ beta(1, 1);
  mu ~ normal(0, 2);
  to_vector(a_raw) ~ normal(0, 1);
  to_vector(b_z) ~ std_normal();
  tau_b ~ normal(0, 0.5);
  s_th ~ normal(0, 1);
  s_ph ~ normal(0, 1);
  s_ps ~ normal(0, 1);
  th_z ~ std_normal();
  to_vector(ph_z) ~ std_normal();
  to_vector(ps_z) ~ std_normal();

  // likelihood, marginalising the latent class
  for (i in 1:N) {
    vector[2] lp = [log1m(pi1), log(pi1)]';
    for (m in 1:M) {
      for (p in 1:P) {

```

```

        real u = s_th * th_z[i] + s_ph * ph_z[i, m] + s_ps * ps_z
        [i, p];
        for (k in 1:2)
            lp[k] += binomial_logit_lpmf(S[i, m, p] | R, mu[k] + a[
            m, k] + b[p, k] + u);
        }
    }
    target += log_sum_exp(lp);
}

generated_quantities {
    vector[N] post1; // P(c_i = 1 | data)
    vector[N] log_lik; // pointwise (per item)
    log-likelihood for L00
    // variance shares on the latent logistic scale (run-level
    residual = pi^2/3)
    real v_res = square(pi()) / 3;
    real v_tot = square(s_th) + square(s_ph) + square(s_ps) + v_res
    ;
    real share_item = square(s_th) / v_tot;
    real share_model = square(s_ph) / v_tot;
    real share_prompt = square(s_ps) / v_tot;
    real share_run = v_res / v_tot;
    for (i in 1:N) {
        vector[2] lp = [log1m(pi1), log(pi1)]';
        for (m in 1:M) for (p in 1:P) {
            real u = s_th * th_z[i] + s_ph * ph_z[i, m] + s_ps * ps_z[i
            , p];
            for (k in 1:2)
                lp[k] += binomial_logit_lpmf(S[i, m, p] | R, mu[k] + a[m,
                k] + b[p, k] + u);
        }
        log_lik[i] = log_sum_exp(lp);
        post1[i] = exp(lp[2] - log_lik[i]);
    }
}

```

References